

Experiment 'p_6_2_e_remove_middle_seq' Results

June 5, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 1

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_6_2_e_remove_middle_seq

Natural language statement: null

Method signature: p_6_2_e_remove_middle_seq(inputs: seq<int>) returns (result: seq<int>)

Ensures

- `|result| == if |inputs| % 2 == 1 then |inputs| - 1 else |inputs| - 2`
- `|inputs| % 2 == 1 ==> forall i :: 0 <= i < |inputs| / 2 ==> result[i] == inputs[i] && forall j :: |inputs| / 2 <= j < |result| ==> result[j] == inputs[j + 1]`
- `|inputs| % 2 == 0 ==> forall i :: 0 <= i < |inputs| / 2 - 1 ==> result[i] == inputs[i] && forall j :: |inputs| / 2 - 1 <= j < |result| ==> result[j] == inputs[j + 2]`

Requires

- `|inputs| >= 1`

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

`p_6_2_e_remove_middle_seq(inputs: seq<int>) returns (result: seq<int>)`

The method should respect the following contract:

```
requires |inputs| >= 1, ensures |result| == if |inputs| % 2 == 1 then |inputs| - 1 else |
  ↪ inputs| - 2, ensures |inputs| % 2 == 1 ==> forall i :: 0 <= i < |inputs| / 2 ==>
  ↪ result[i] == inputs[i] && forall j :: |inputs| / 2 <= j < |result| ==> result[j]
  ↪ == inputs[j + 1], ensures |inputs| % 2 == 0 ==> forall i :: 0 <= i < |inputs| / 2
  ↪ - 1 ==> result[i] == inputs[i] && forall j :: |inputs| / 2 - 1 <= j < |result| ==>
  ↪ result[j] == inputs[j + 2]
```

Produce and show only the Dafny body of this method, including the curly braces that
↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
↪ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnNnuMiHOOeb0uwz0fb4H1UhkggfR

Model: gpt-5.5-2026-04-23

Created at: 1780661946

Finish reason: stop

Usage: 566 tokens in, and 7646 tokens out

```
{
  var n := |inputs|;
  if n % 2 == 1 {
    var k := n / 2;
    assert n == (n / 2) * 2 + n % 2;
    assert n == k * 2 + 1;
    assert n == k + (k + 1);
    assert 0 <= k;
    assert 0 <= k + 1 <= n;

    var prefix := inputs[..k];
    var suffix := inputs[k + 1..];
    result := prefix + suffix;

    assert |prefix| == k;
    assert |suffix| == n - (k + 1);
```

```

assert |result| == |prefix| + |suffix|;
assert |result| == n - 1;

forall i | 0 <= i < k
  ensures result[i] == inputs[i]
{
  assert i < |prefix|;
  assert result[i] == prefix[i];
  assert prefix[i] == inputs[i];
}

forall j | k <= j < |result|
  ensures result[j] == inputs[j + 1]
{
  assert |prefix| == k;
  assert j >= |prefix|;
  assert j < |prefix| + |suffix|;
  assert 0 <= j - |prefix| < |suffix|;
  assert result[j] == suffix[j - |prefix|];
  assert j - |prefix| == j - k;
  assert suffix[j - k] == inputs[k + 1 + (j - k)];
  assert k + 1 + (j - k) == j + 1;
}

assert forall i :: 0 <= i < k ==> result[i] == inputs[i];
assert forall j :: k <= j < |result| ==> result[j] == inputs[j + 1];
assert forall i :: 0 <= i < k ==> result[i] == inputs[i] && (forall j :: k <= j < |
  ↪ result| ==> result[j] == inputs[j + 1]);
} else {
  assert 0 <= n % 2 < 2;
  assert n % 2 == 0;
  assert n != 1;
  assert n >= 2;

  var h := n / 2;
  assert n == (n / 2) * 2 + n % 2;
  assert n == h * 2;
  assert n == h + h;
  assert h >= 1;

  var k := h - 1;
  var start := h + 1;
  assert k == n / 2 - 1;
  assert 0 <= k;
  assert 0 <= start <= n;

  var prefix := inputs[..k];
  var suffix := inputs[start..];
  result := prefix + suffix;

  assert |prefix| == k;

```

```

assert |suffix| == n - start;
assert |result| == |prefix| + |suffix|;
assert |result| == n - 2;

forall i | 0 <= i < k
  ensures result[i] == inputs[i]
{
  assert i < |prefix|;
  assert result[i] == prefix[i];
  assert prefix[i] == inputs[i];
}

forall j | k <= j < |result|
  ensures result[j] == inputs[j + 2]
{
  assert |prefix| == k;
  assert j >= |prefix|;
  assert j < |prefix| + |suffix|;
  assert 0 <= j - |prefix| < |suffix|;
  assert result[j] == suffix[j - |prefix|];
  assert j - |prefix| == j - k;
  assert suffix[j - k] == inputs[start + (j - k)];
  assert start + (j - k) == j + 2;
}

assert forall i :: 0 <= i < k ==> result[i] == inputs[i];
assert forall j :: k <= j < |result| ==> result[j] == inputs[j + 2];
assert forall i :: 0 <= i < k ==> result[i] == inputs[i] && (forall j :: k <= j < |
  ↪ result| ==> result[j] == inputs[j + 2]);
}
}

```

Final Program

```

method p_6_2_e_remove_middle_seq(inputs: seq<int>) returns (result: seq<int>)
  requires |inputs| >= 1
  ensures |result| == if |inputs| % 2 == 1 then |inputs| - 1 else |inputs| - 2
  ensures |inputs| % 2 == 1 ==> forall i :: 0 <= i < |inputs| / 2 ==> result[i] ==
    ↪ inputs[i] && forall j :: |inputs| / 2 <= j < |result| ==> result[j] ==
    ↪ inputs[j + 1]
  ensures |inputs| % 2 == 0 ==> forall i :: 0 <= i < |inputs| / 2 - 1 ==> result[i]
    ↪ == inputs[i] && forall j :: |inputs| / 2 - 1 <= j < |result| ==> result[j]
    ↪ == inputs[j + 2]
{
  var n := |inputs|;
  if n % 2 == 1 {
    var k := n / 2;
    assert n == (n / 2) * 2 + n % 2;
    assert n == k * 2 + 1;
    assert n == k + (k + 1);
  }
}

```

```

assert 0 <= k;
assert 0 <= k + 1 <= n;

var prefix := inputs[..k];
var suffix := inputs[k + 1..];
result := prefix + suffix;

assert |prefix| == k;
assert |suffix| == n - (k + 1);
assert |result| == |prefix| + |suffix|;
assert |result| == n - 1;

forall i | 0 <= i < k
  ensures result[i] == inputs[i]
{
  assert i < |prefix|;
  assert result[i] == prefix[i];
  assert prefix[i] == inputs[i];
}

forall j | k <= j < |result|
  ensures result[j] == inputs[j + 1]
{
  assert |prefix| == k;
  assert j >= |prefix|;
  assert j < |prefix| + |suffix|;
  assert 0 <= j - |prefix| < |suffix|;
  assert result[j] == suffix[j - |prefix|];
  assert j - |prefix| == j - k;
  assert suffix[j - k] == inputs[k + 1 + (j - k)];
  assert k + 1 + (j - k) == j + 1;
}

assert forall i :: 0 <= i < k ==> result[i] == inputs[i];
assert forall j :: k <= j < |result| ==> result[j] == inputs[j + 1];
assert forall i :: 0 <= i < k ==> result[i] == inputs[i] && (forall j :: k <= j < |
  ↪ result| ==> result[j] == inputs[j + 1]);
} else {
  assert 0 <= n % 2 < 2;
  assert n % 2 == 0;
  assert n != 1;
  assert n >= 2;

  var h := n / 2;
  assert n == (n / 2) * 2 + n % 2;
  assert n == h * 2;
  assert n == h + h;
  assert h >= 1;

  var k := h - 1;
  var start := h + 1;

```

```

assert k == n / 2 - 1;
assert 0 <= k;
assert 0 <= start <= n;

var prefix := inputs[..k];
var suffix := inputs[start..];
result := prefix + suffix;

assert |prefix| == k;
assert |suffix| == n - start;
assert |result| == |prefix| + |suffix|;
assert |result| == n - 2;

forall i | 0 <= i < k
  ensures result[i] == inputs[i]
{
  assert i < |prefix|;
  assert result[i] == prefix[i];
  assert prefix[i] == inputs[i];
}

forall j | k <= j < |result|
  ensures result[j] == inputs[j + 2]
{
  assert |prefix| == k;
  assert j >= |prefix|;
  assert j < |prefix| + |suffix|;
  assert 0 <= j - |prefix| < |suffix|;
  assert result[j] == suffix[j - |prefix|];
  assert j - |prefix| == j - k;
  assert suffix[j - k] == inputs[start + (j - k)];
  assert start + (j - k) == j + 2;
}

assert forall i :: 0 <= i < k ==> result[i] == inputs[i];
assert forall j :: k <= j < |result| ==> result[j] == inputs[j + 2];
assert forall i :: 0 <= i < k ==> result[i] == inputs[i] && (forall j :: k <= j < |
  ↪ result| ==> result[j] == inputs[j + 2]);
}
}

```

Total Token Usage

Input tokens: 566

Output tokens: 7646

Reasoning tokens: 6547

Sum of 'total tokens': 8212

Experiment Timings

Overall Experiment started at 1780661945622, ended at 1780662105537, lasting 159915ms (159.92 seconds)

Iteration #1 started at 1780661945623, ended at 1780662105523, lasting 159900ms (159.90 seconds)